

Introductory Project (1DT308)

IoT Communication & MQTT

Arslan Musaddiq¹

Faculty of Technology

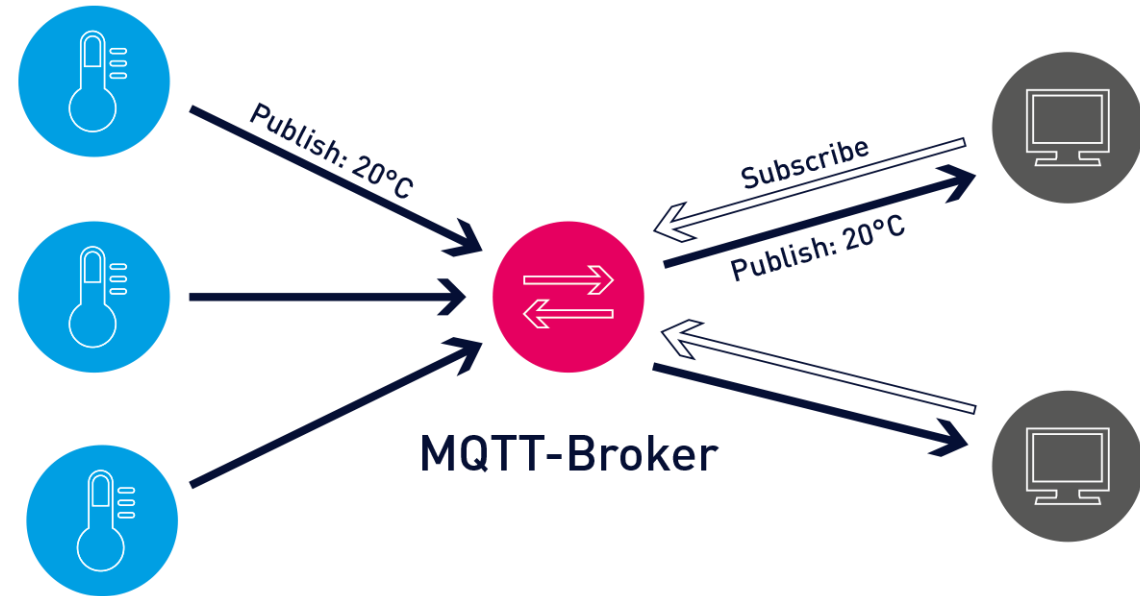
Department of Computer science and Media Technology

Linnaeus University, Sweden.



Message Queuing Telemetry Transport (MQTT)

- MQTT stands for **Message Queuing Telemetry Transport**,
 - a simple messaging protocol, designed for constrained devices with low bandwidth and is suitable for communication between IoT devices.
- MQTT communication works as a **publish and subscribe system** (introduced by IBM in 1999).
- Devices publish messages on a specific topic. All devices that are subscribed to that topic receive the message.



MQTT Components:

- Publisher (such as sensors).
- Subscriber (applications interested in the data).
- Broker (connecting publisher and subscriber).

MQTT Methods:

- connect, disconnect, publish, subscribe



<https://mqtt.org/>



MQTT Working Principle

- MQTT uses a topic-based messaging model.
- Devices (publishers) send messages to specific topics.
- Other devices (subscribers) receive these messages if they are subscribed to the same topic.



Quality of Service (QoS)

- It supports three levels of Quality of Service (QoS):
 - QoS 0 (at most once): The broker/client will deliver the message once, with no confirmation.
 - QoS 1 (at least once): The broker/client will deliver the message at least once, with the confirmation.
 - QoS 2: (exactly once): The broker/client will deliver the message exactly once by using a four-step handshake.



MQTT in Real-Life Scenarios

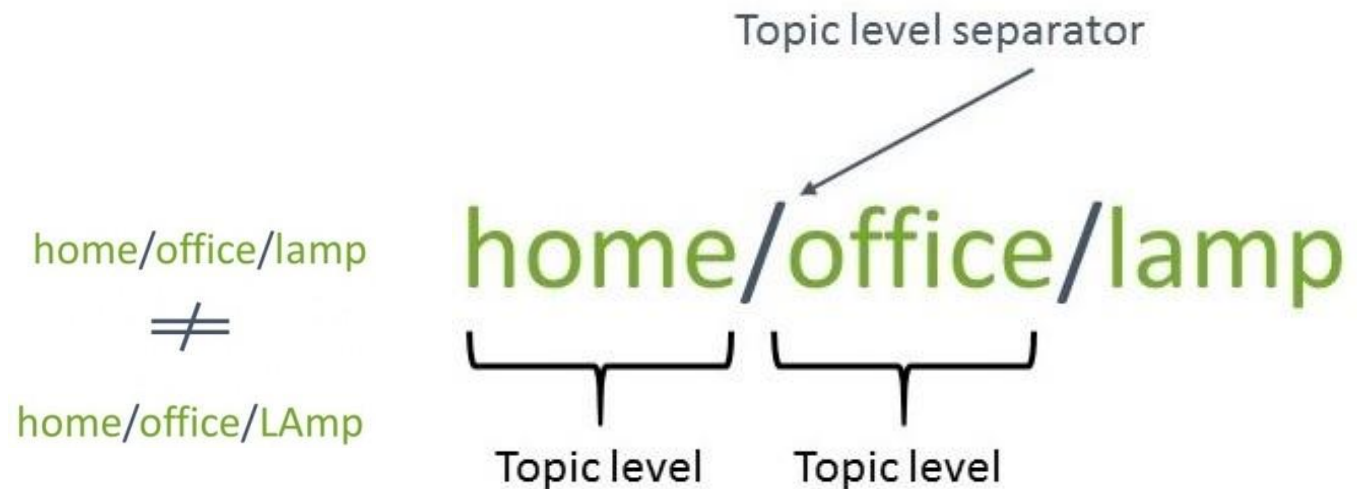
1. A weather station publishes temperature and humidity data to an MQTT broker on a specific topic.
2. This data is subscribed to by an IoT device that controls a greenhouse's temperature and humidity levels.
3. The IoT device receives the data and uses it to make decisions about when to turn on or off the greenhouse's heating and cooling systems.

1. A smart home hub publishes commands to turn off the lights in the living room to an MQTT broker on a specific topic.
2. This command is then subscribed to by a smart light bulb.
3. The light bulb receives the command and turns off the lights in response.



MQTT-Topics

- Topics are the way you register interest in incoming messages or how you specify where you want to publish the message.
- Topics are represented with strings separated by a forward slash.
- Each forward slash indicates a topic level.
- Topics are case-sensitive.



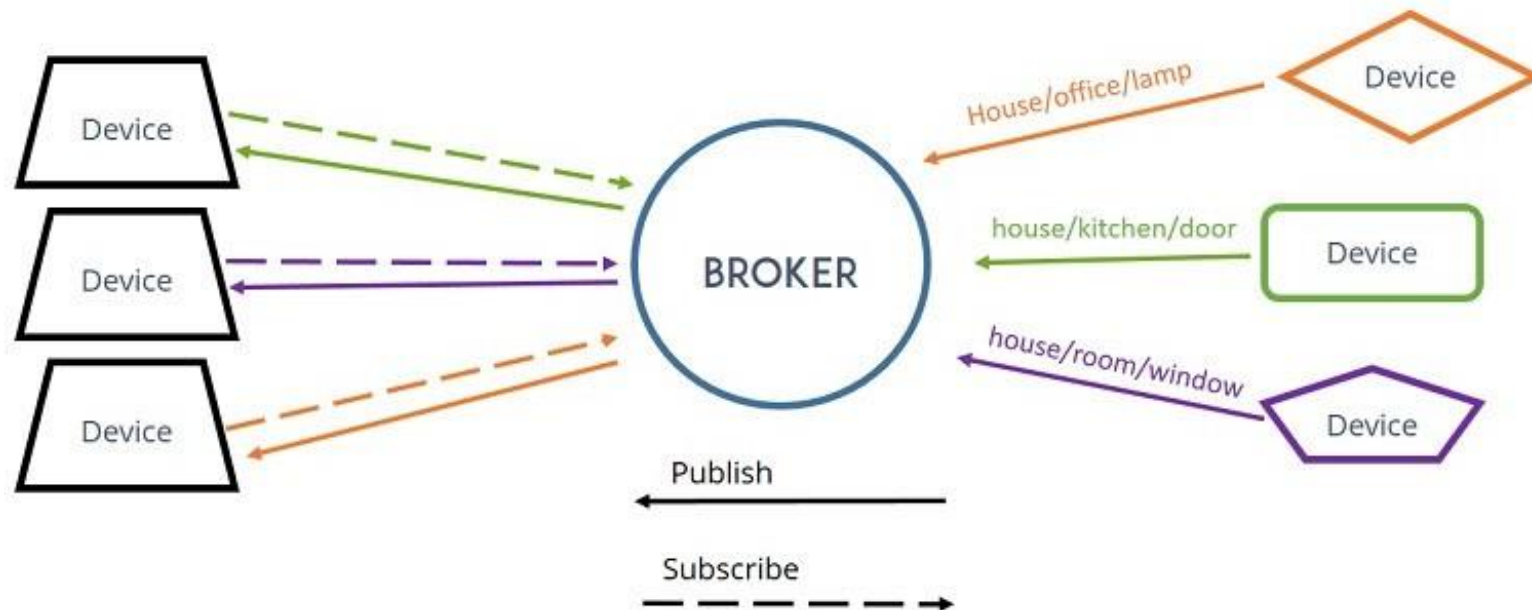
✓ An example of how you would create a topic for a lamp in your home office.



MQTT-Broker

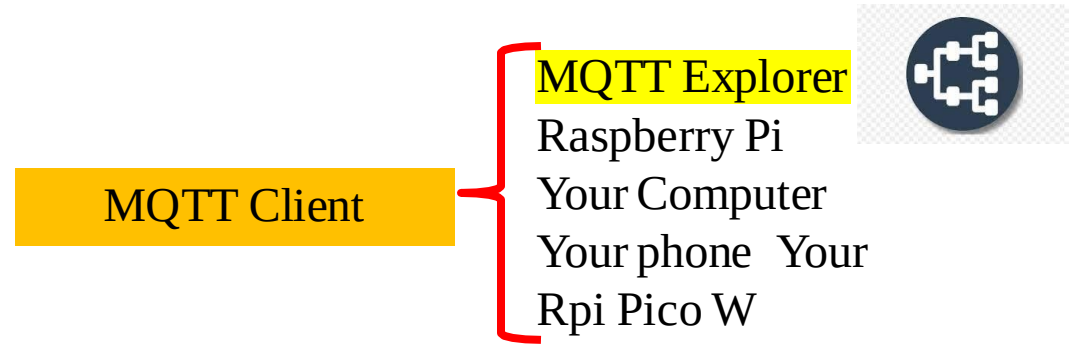
The **MQTT broker** is responsible for:

- **receiving** all messages,
- **filtering** the messages,
- **deciding** who is interested in them
- **publishing** the message to all subscribed clients.

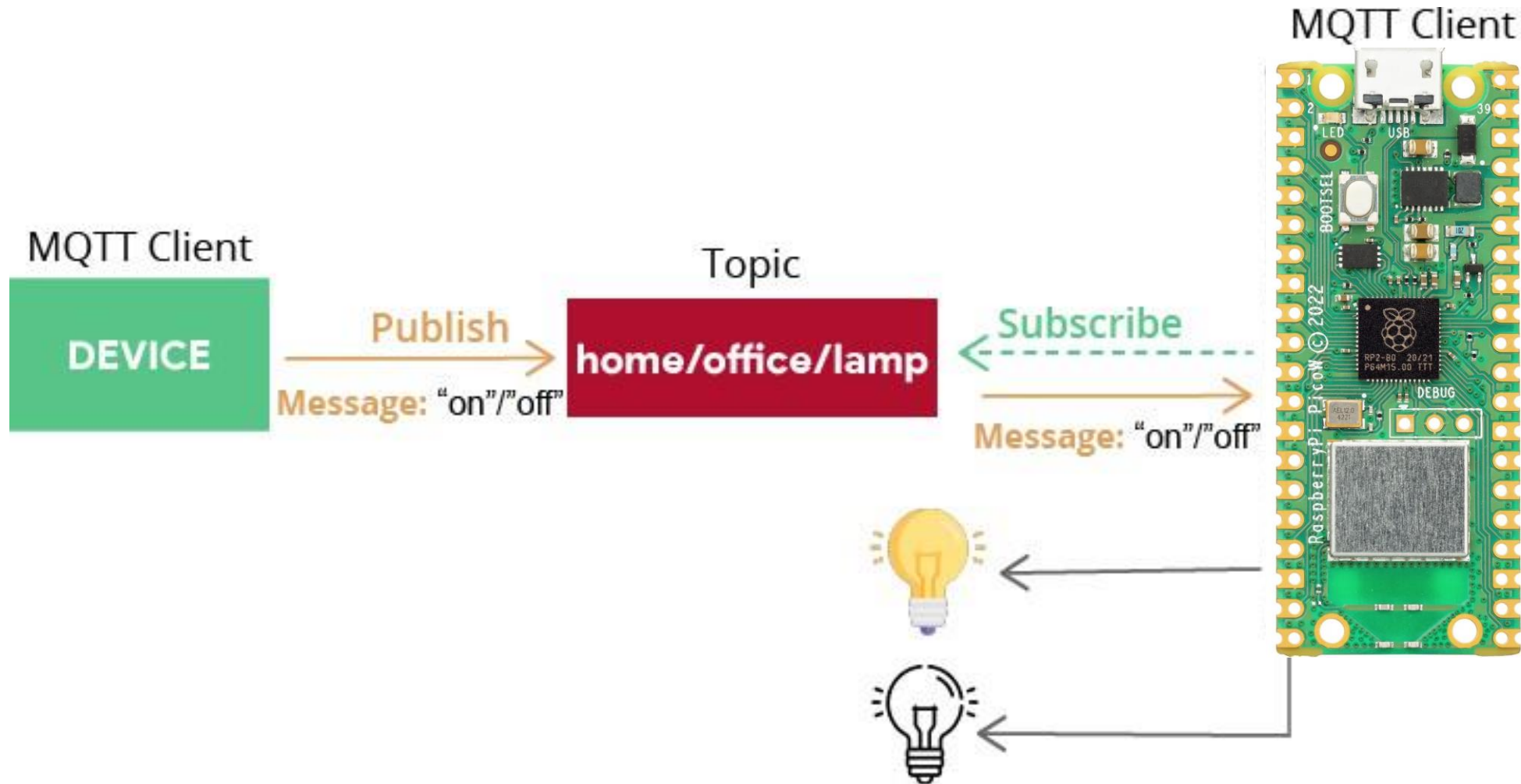


MQTT-Client

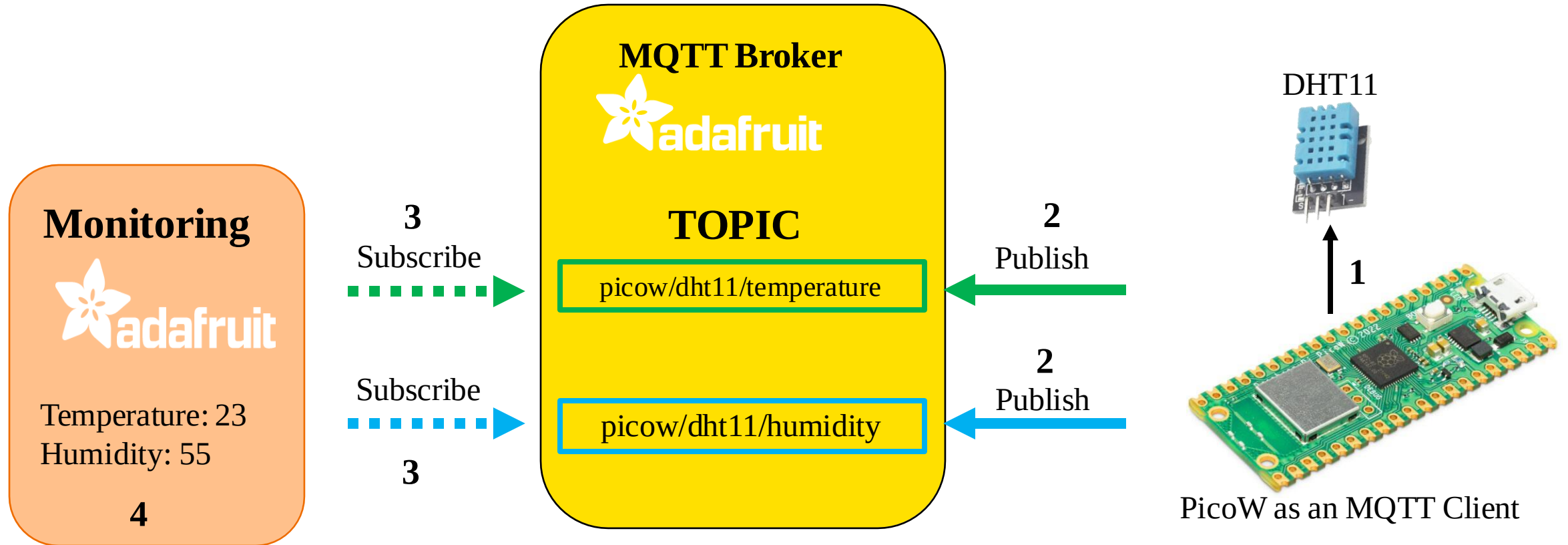
- An MQTT client is a software application or device that connects to an MQTT broker to send or receive messages.
- MQTT clients can be implemented on a wide range of platforms, from embedded devices to desktop computers and servers.
- MQTT Explorer is a client tool that is used to explore and debug MQTT-based applications.
- MQTT Explorer is not a stand-alone entity and it needs to work with one of the MQTT brokers.



Example of Remote Controlling: Home Automation



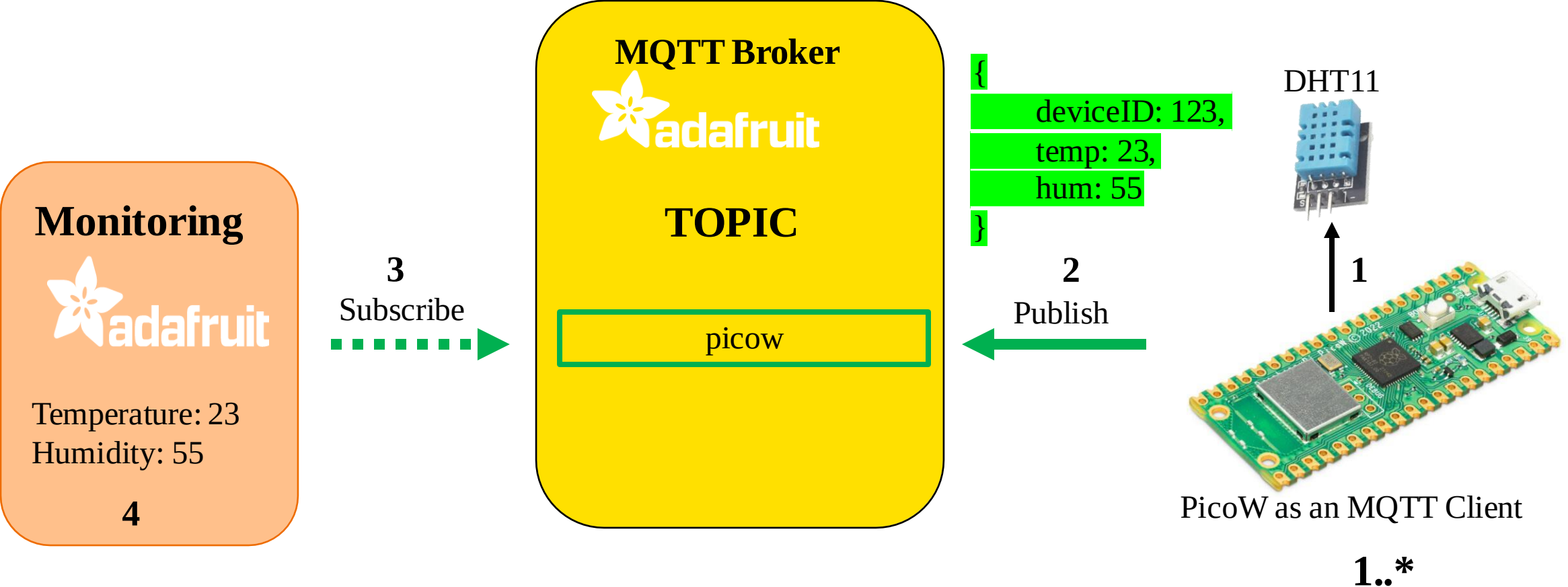
Example of Remote Sensing



- ✓ MQTT Broker can be any from MQTT_Brokers list.
- ✓ Monitoring and visualization can be either any from the MQTT_Brokers list or MQTTExplorer.



Project: A High-level Overview



- ✓ MQTT Broker can be any from MQTT_Brokers list.
- ✓ Monitoring and visualization can be either any from the MQTT_Brokers list or MQTTExplorer.

Free MQTT Brokers

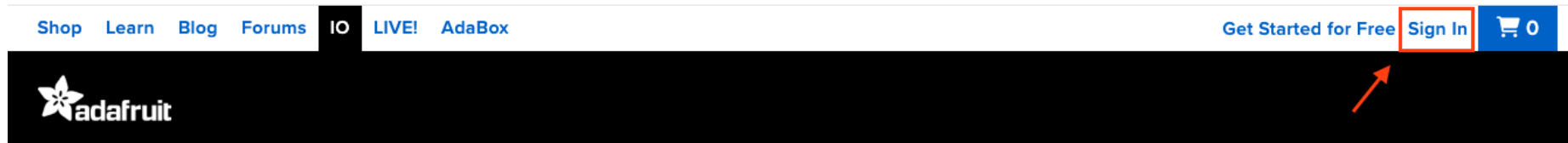
- Eclipse Mosquitto
<https://mosquitto.org/>
- HiveMQ Community Edition (HiveMQ CE)
<https://www.hivemq.com/>
- EMQX MQTT Broker
<https://www.emqx.com/en/mqtt/public-mqtt5-broker>
- ThingSpeak MQTT Support
<https://thingspeak.com>
- Adafruit IO
<https://io.adafruit.com>



Setup the MQTT Server/Broker (Adafruit IO)

1. Go to the Adafruit website using the following link:

<https://www.adafruit.com>



Setup the MQTT Server/Broker (Adafruit IO)

2. Create an account on the Adafruit website:

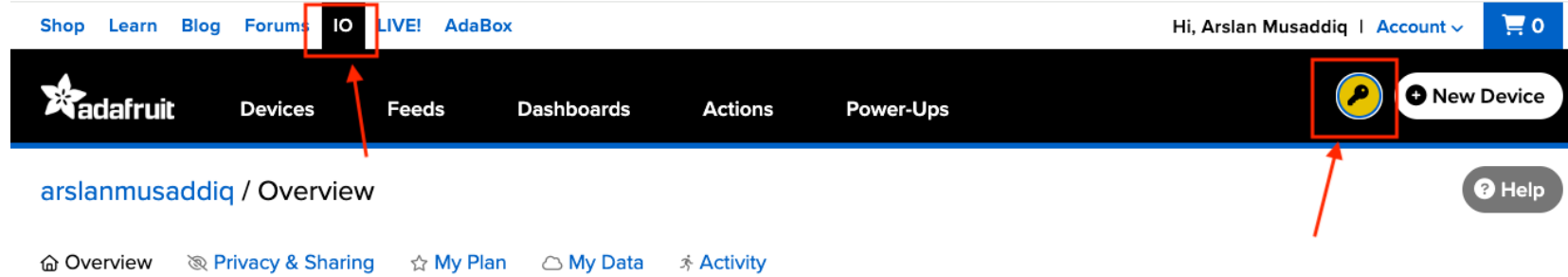
The screenshot shows the Adafruit website's header with navigation links: Shop, Learn, Blog, Forums, IO, LIVE!, and AdaBox. On the right, there is a 'Sign In' link and a shopping cart icon with '0' items. Below the header is a black banner with the Adafruit logo. The main content area is divided into two columns. The left column is titled 'Sign In' and contains a description of the benefits of an account, input fields for 'Email or Username' and 'Password', a blue 'Sign In' button, and a link for 'Forgot your password?'. The right column is titled 'Order Status' and contains a description, input fields for 'Email Address' and 'Order Number', a dark grey 'Check Order Status' button, and a link for 'Where do I find the order number?'. At the bottom of the 'Sign In' column, there is a button labeled 'Create an Adafruit Account', which is highlighted with a red rectangular box. A red arrow points from the right towards this button.

Setup the MQTT Server/Broker (Adafruit IO)

3. You will end up in the following URL:

<https://accounts.adafruit.com/>

4. Go to the IO tab, then click on the yellow key:



Setup the MQTT Server/Broker (Adafruit IO)

5. Generate Key for your username:

YOUR ADAFRUIT IO KEY ✕

Your Adafruit IO Key should be kept in a safe place and treated with the same care as your Adafruit username and password. People who have access to your Adafruit IO Key can view all of your data, create new feeds for your account, and manipulate your active feeds.

If you need to regenerate a new Adafruit IO Key, all of your existing programs and scripts will need to be manually changed to the new key.

Username

arslanmusaddiq

Active Key

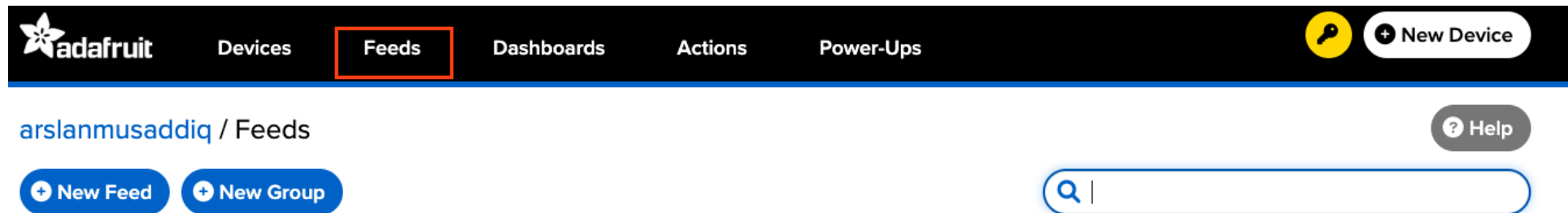
aio_FukD49XFTEG719Dpnb8L5wWSOKyb

REGENERATE KEY



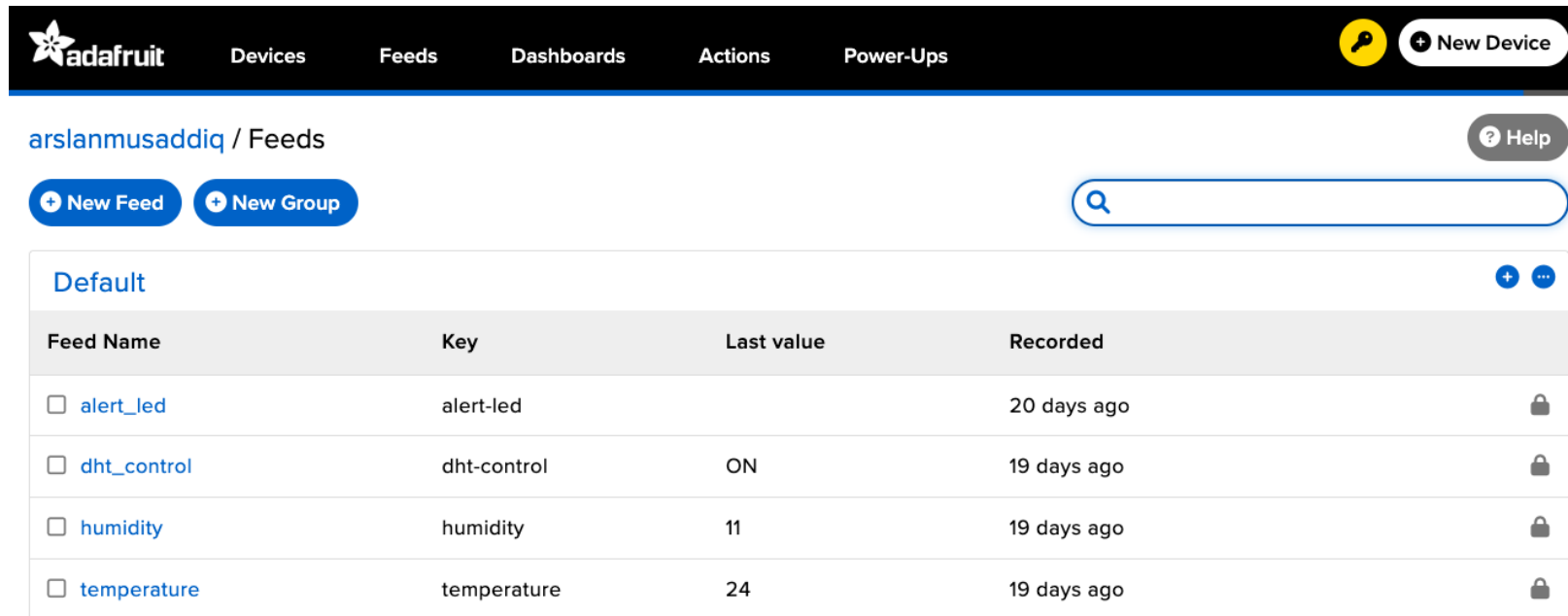
Create a Topic on Adafruit

- **Login** into your Adafruit account.
- Go to the **IO** tab
- Choose the **Feeds** tab
- Choose the **New Feed** button



Create a Topic on Adafruit

- Enter a name for your feed, for example: *temperature*
- You can set the permissions for your feed, such as whether it can be publicly or privately accessible (Feeds are private by default).



The screenshot shows the Adafruit web interface for managing feeds. The top navigation bar includes links for Devices, Feeds, Dashboards, Actions, and Power-Ups, along with a 'New Device' button. The user 'arslanmusaddiq' is logged in. Below the navigation bar, there are buttons for 'New Feed' and 'New Group', and a search bar. A table titled 'Default' lists existing feeds with columns for Feed Name, Key, Last value, and Recorded time. Each feed has a checkbox and a lock icon.

Feed Name	Key	Last value	Recorded
☐ alert_led	alert-led		20 days ago
☐ dht_control	dht-control	ON	19 days ago
☐ humidity	humidity	11	19 days ago
☐ temperature	temperature	24	19 days ago



Two Common Data Serialization Formats

- JSON

- is a lightweight data-interchange format commonly used in IoT applications.
- is a useful way to store and exchange data between different applications or systems.
- is easy for humans to read and write, and easy for machines to parse and generate.
- It uses a simple text-based syntax to represent data structures consisting of key-value pairs, arrays, and nested objects.

```
{  
  "thing": 1,  
  "temperature": 23.45,  
  "pressure": 1023,  
  "humidity": 44,  
  "gas": 445,  
  "ds18x20_one": 23.4,  
  "ds18x20_two": 22.8,  
  "weight": 14  
}
```

- Ordered Bytes

- is a binary data serialization format that is optimized for performance and space efficiency.
- It represents data structures as a sequence of bytes in a specific order, with each data type represented by a fixed number of bytes.

```
b' \x00\x01\t) \x03\xff, \x01\xc2\t$\x08\xe8  
 \x0e'
```





Lnu.se